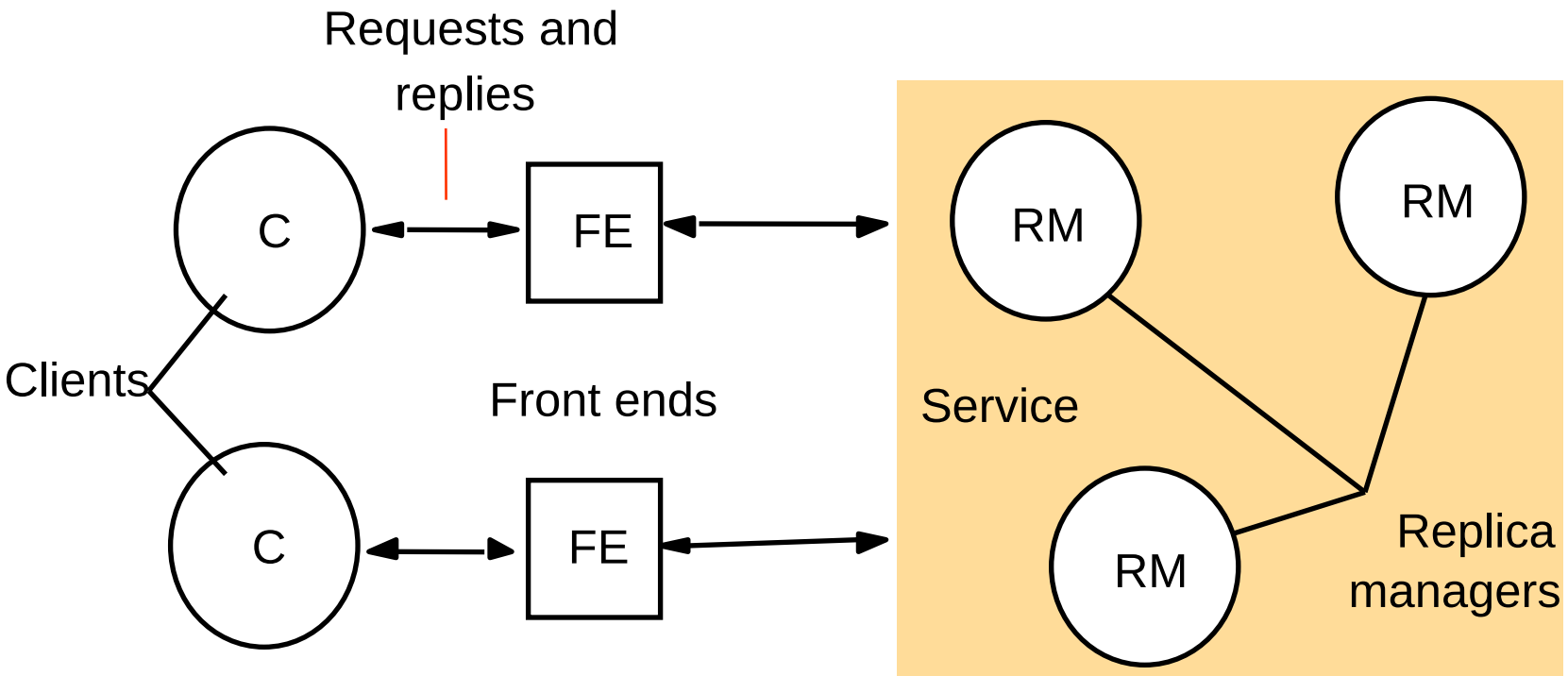
Distributed Systems
Replication

Pedro García López
pedro.garcia@urv.cat

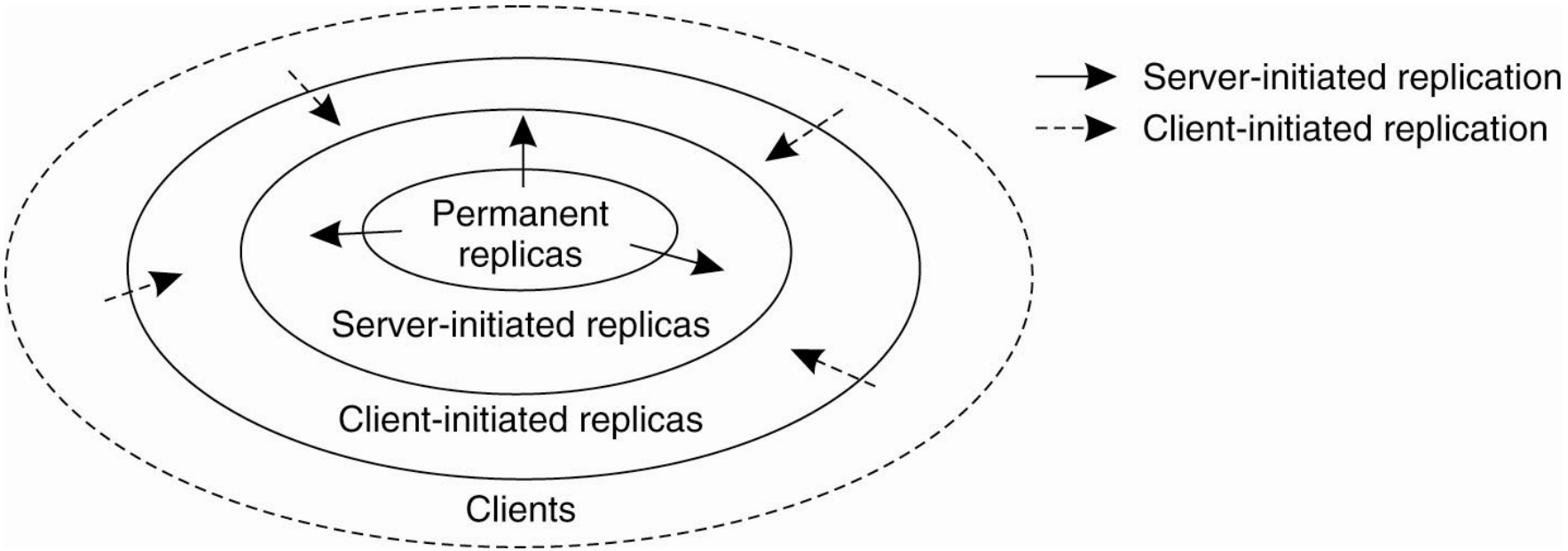
Copyright

A basic architectural model for the management of replicated data

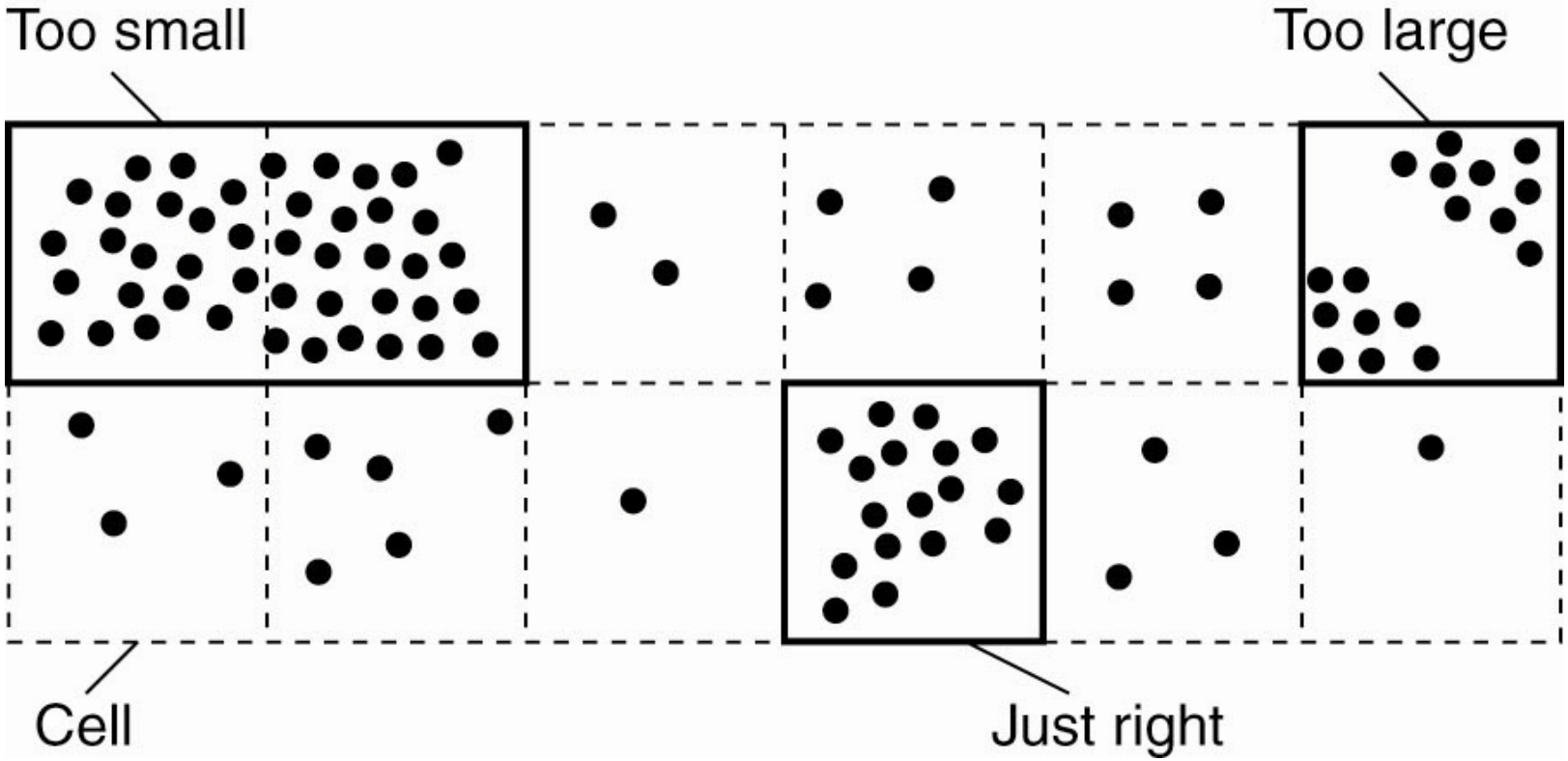


Content Replication and Placement

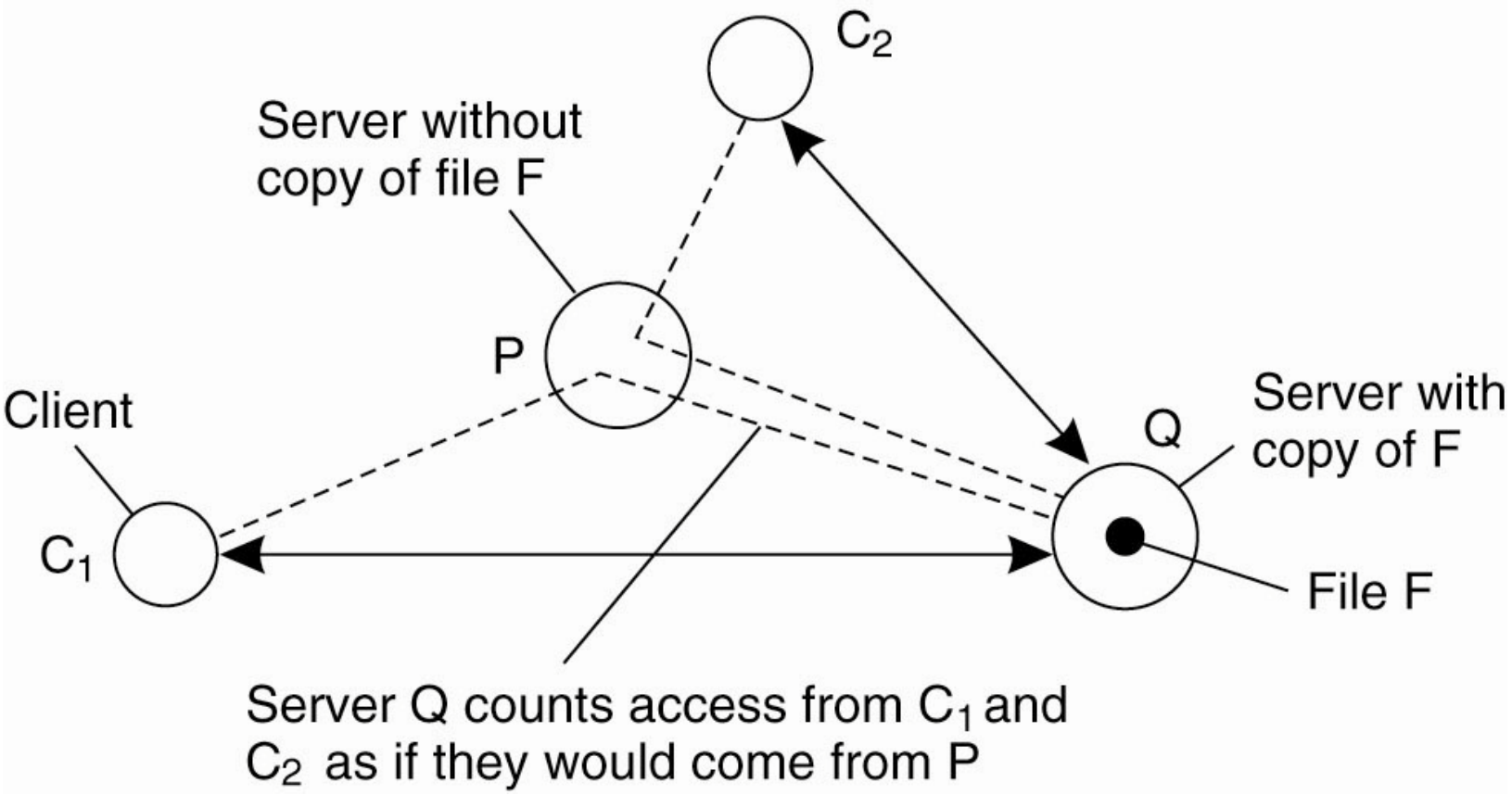
- The logical organization of different kinds of copies of a data store into three concentric rings.



Replica-Server Placement



Server-Initiated Replicas



- Counting access requests from different clients.

State versus Operations

- Possibilities for what is to be propagated:

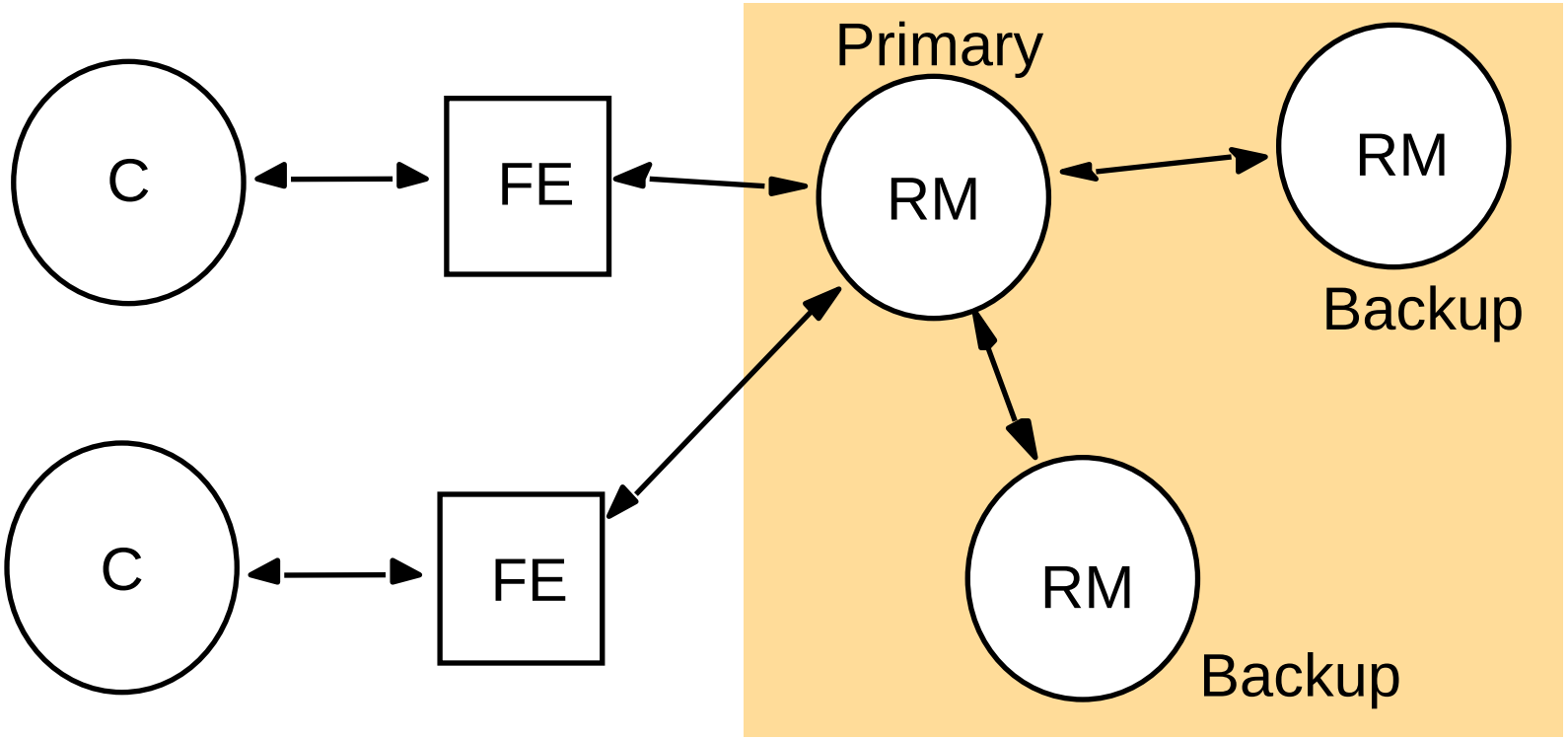
1. Propagate only a notification of an update.
2. Transfer data from one copy to another.
3. Propagate the update operation to other copies.

Pull versus Push Protocols

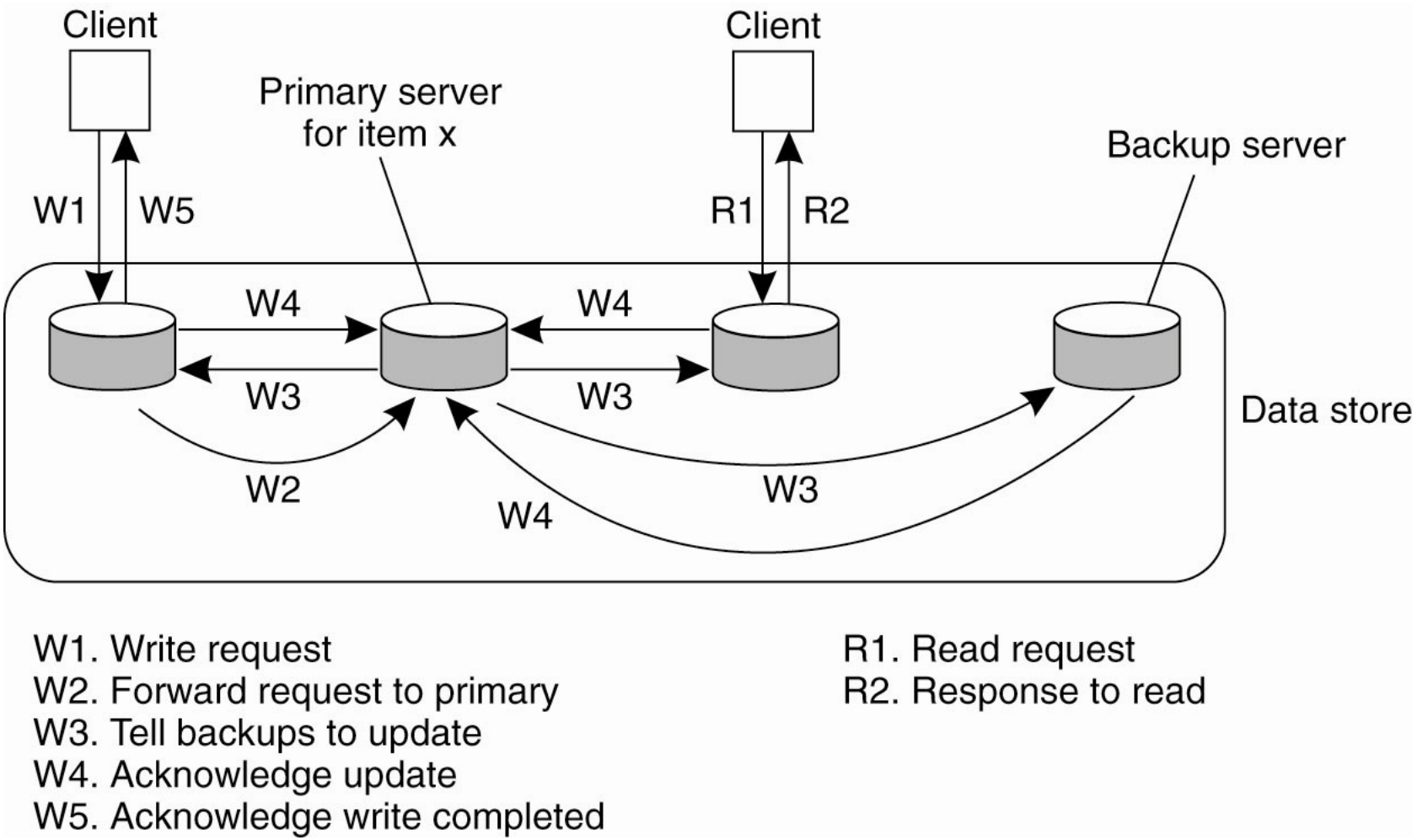
Issue	Push-based	Pull-based
State at server	List of client replicas and caches	None
Messages sent	Update (and possibly fetch update later)	Poll and update
Response time at client	Immediate (or fetch-update time)	Fetch-update time

- A comparison between push-based and pull-based protocols in the case of multiple-client, single-server systems.

The passive (primary-backup) model for fault tolerance

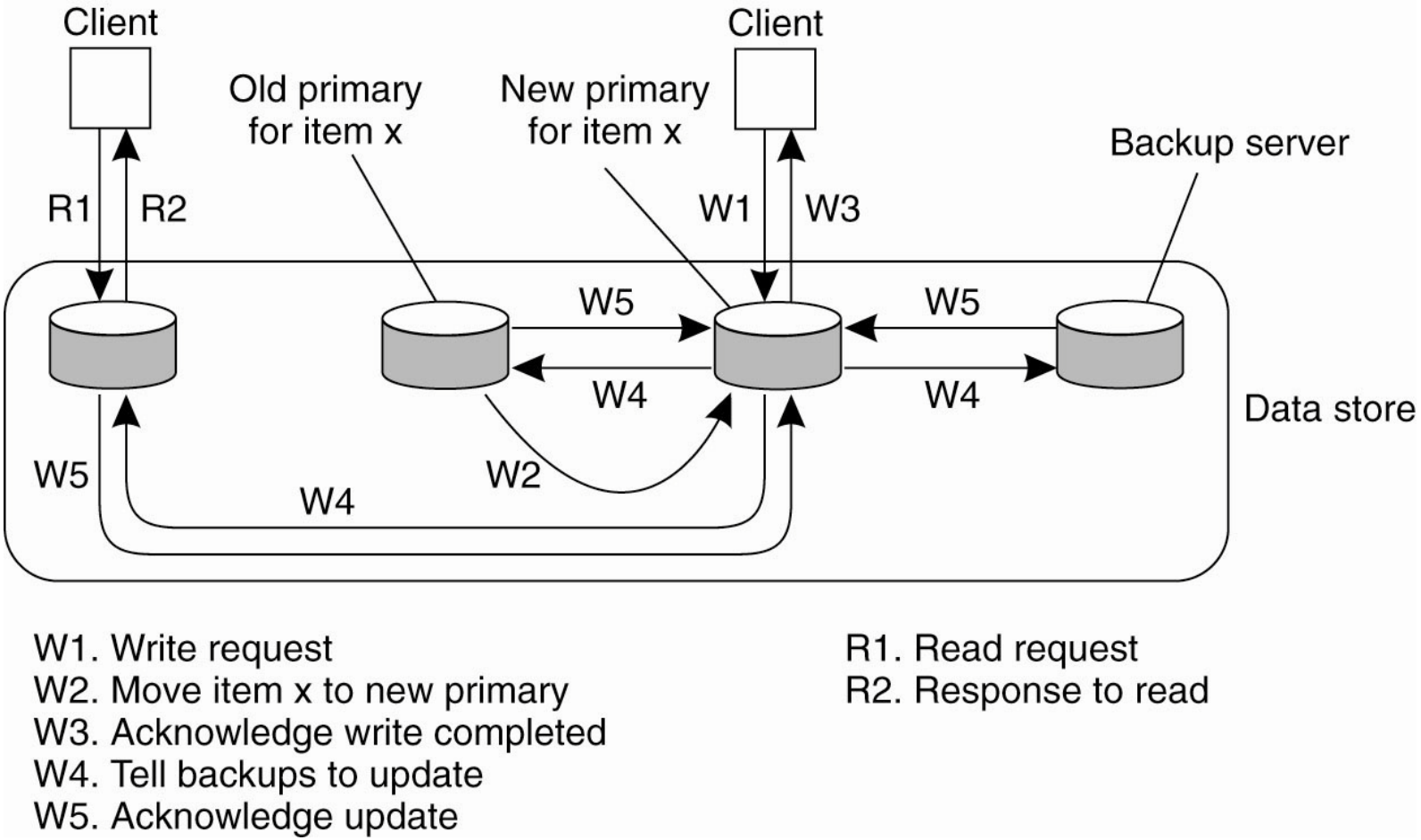


Remote-Write Protocols



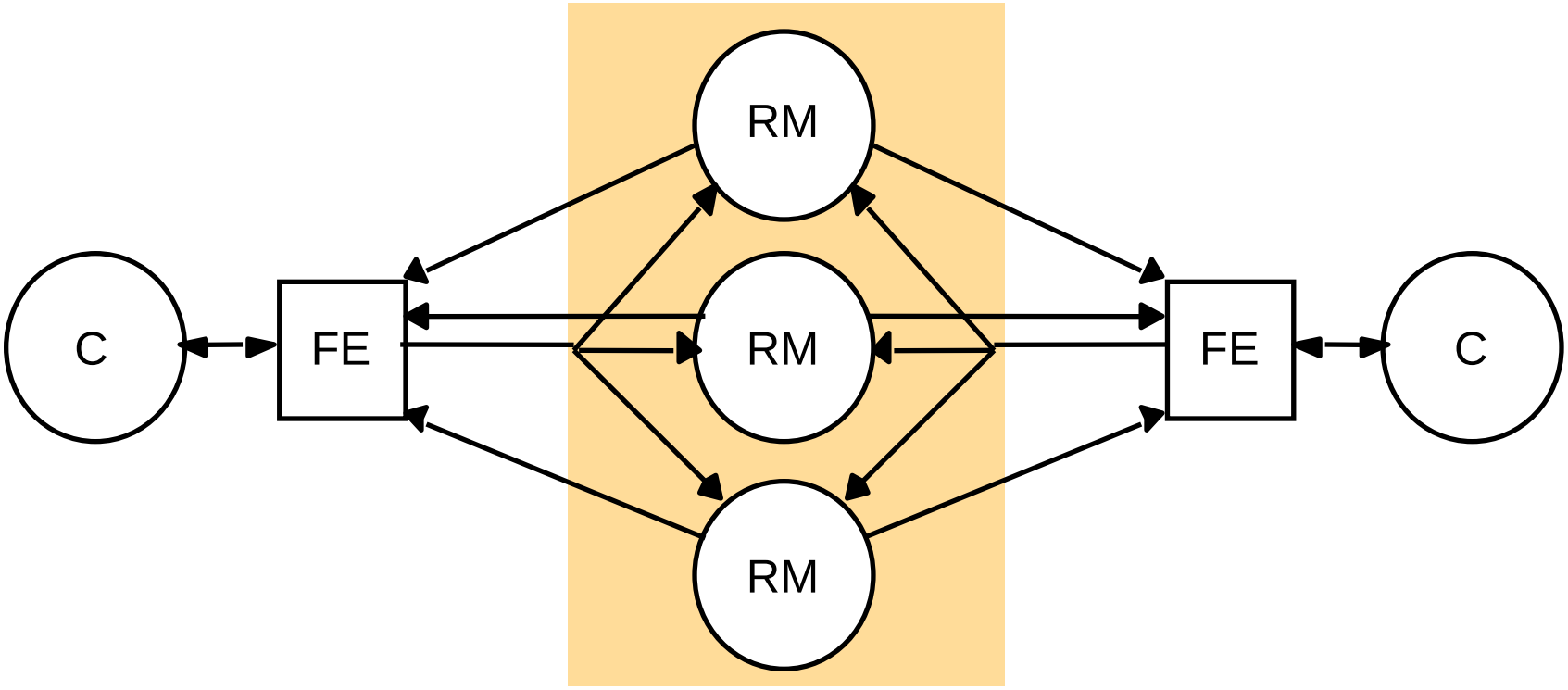
- The principle of a primary-backup protocol.

Local-Write Protocols



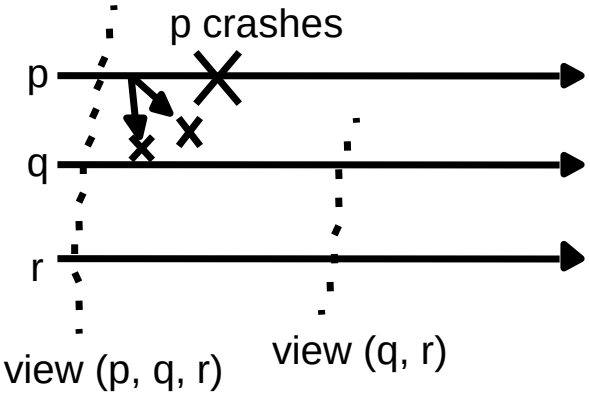
- Primary-backup protocol in which the primary migrates to the process wanting to perform an update.

Active replication

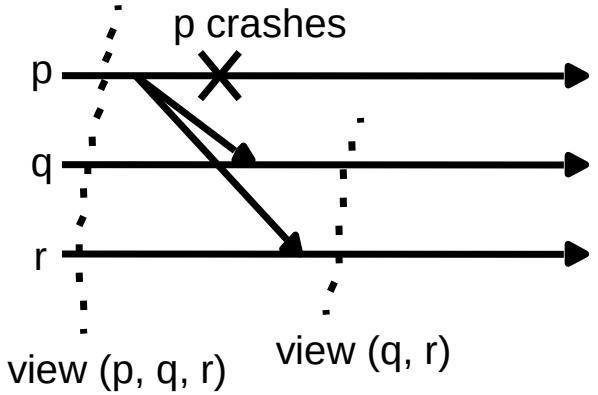


View-synchronous group communication

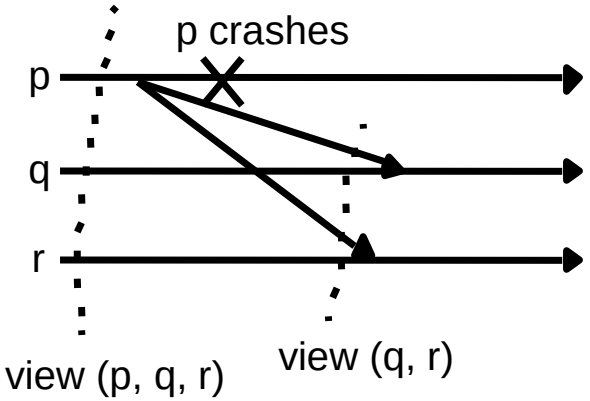
a (allowed).



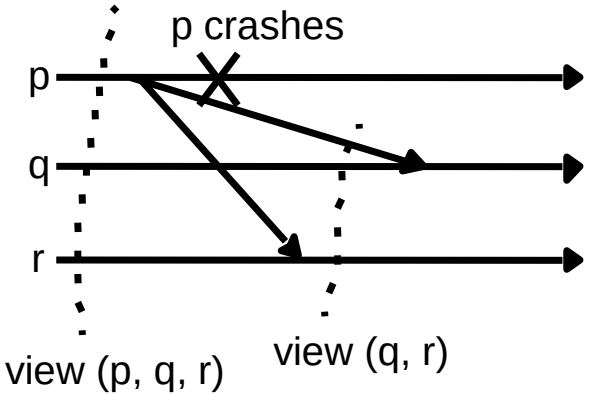
b (allowed).



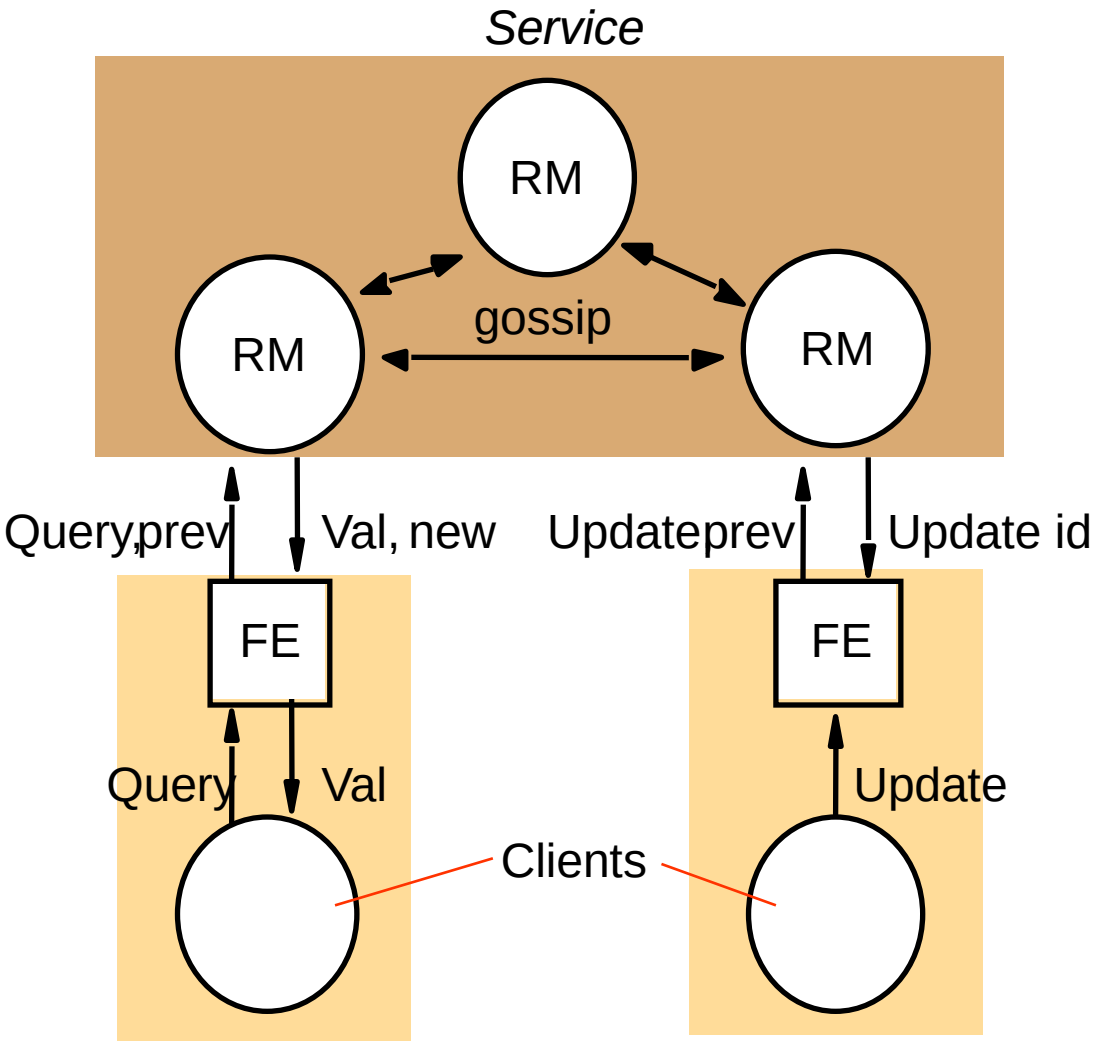
c (disallowed).



d (disallowed).

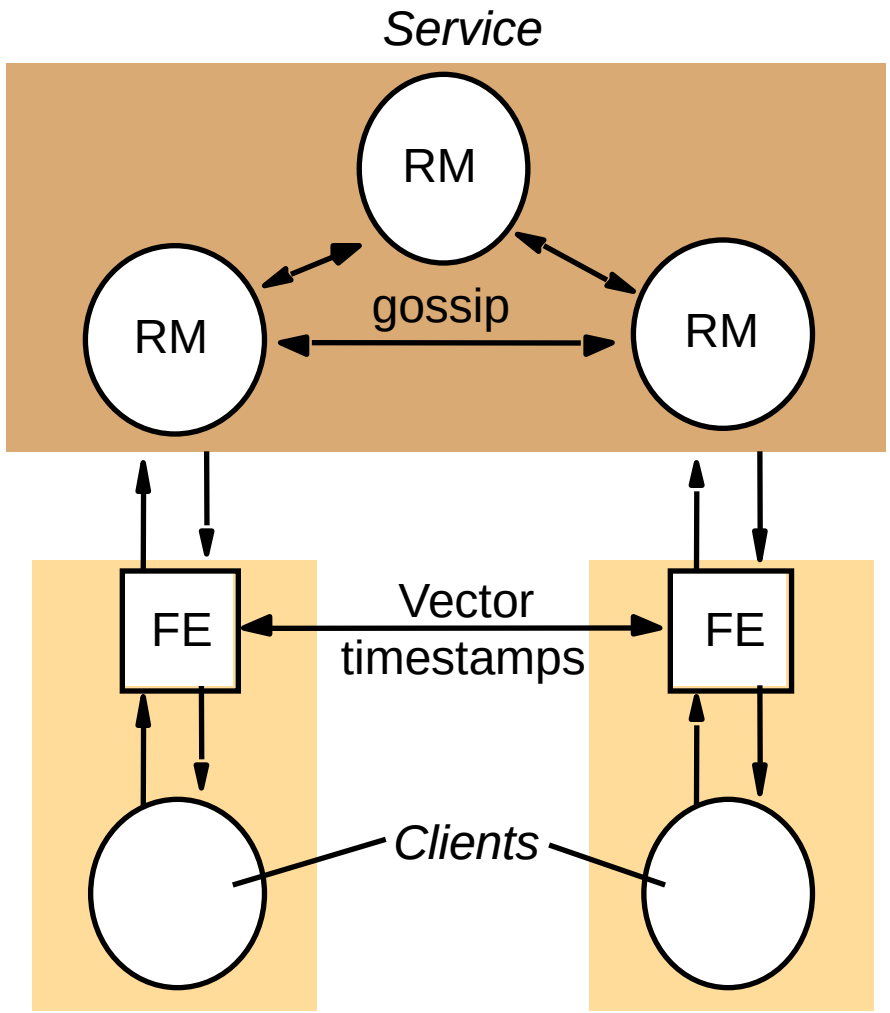


Query and update operations in a gossip service

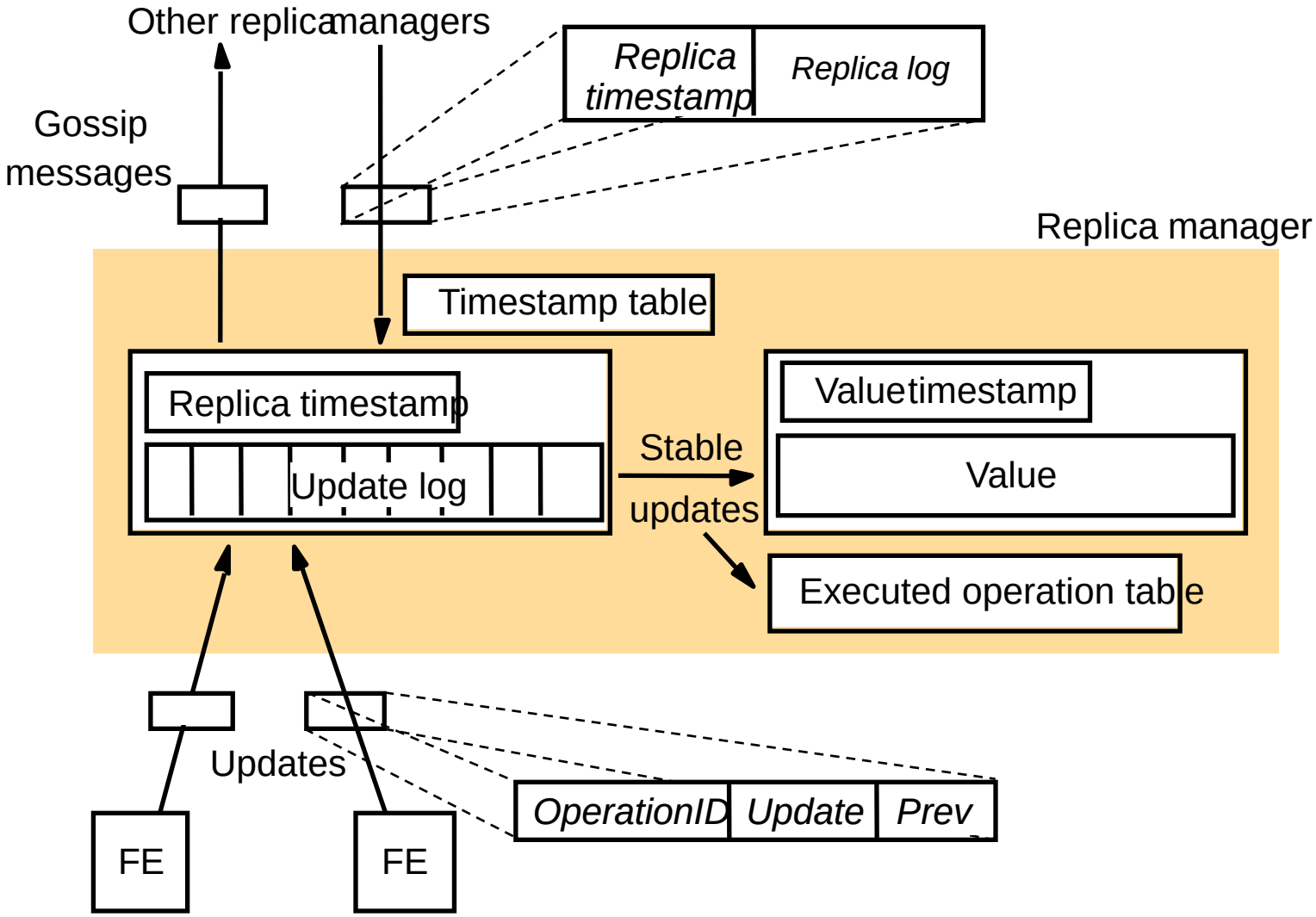


Gossip Architecture

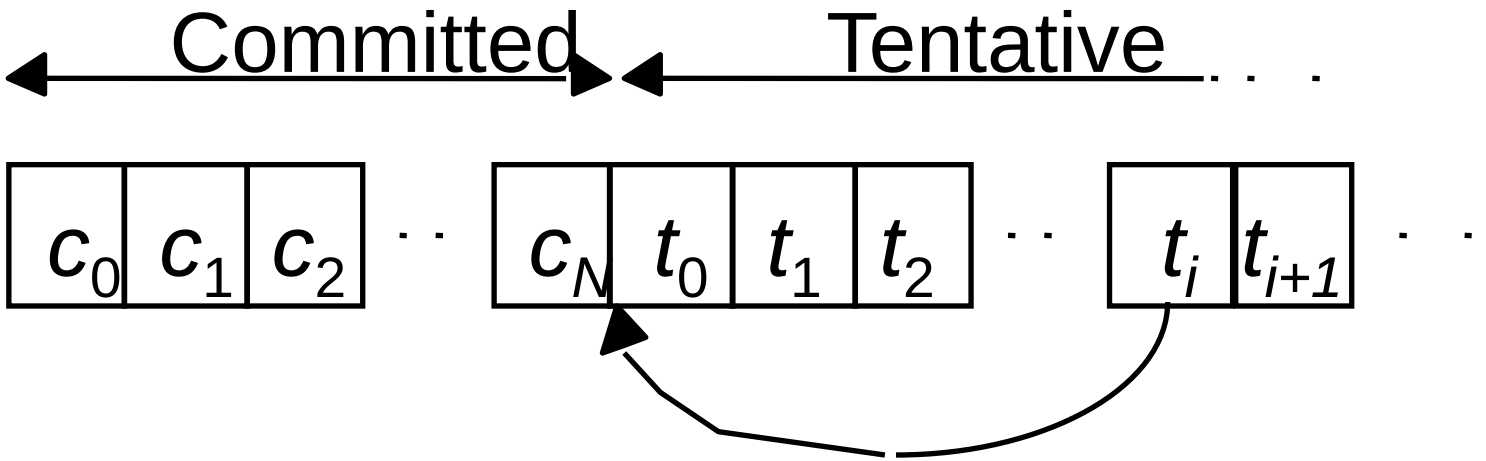
Front ends propagate their timestamps whenever clients communicate directly



A gossip replica manager, showing its main state components

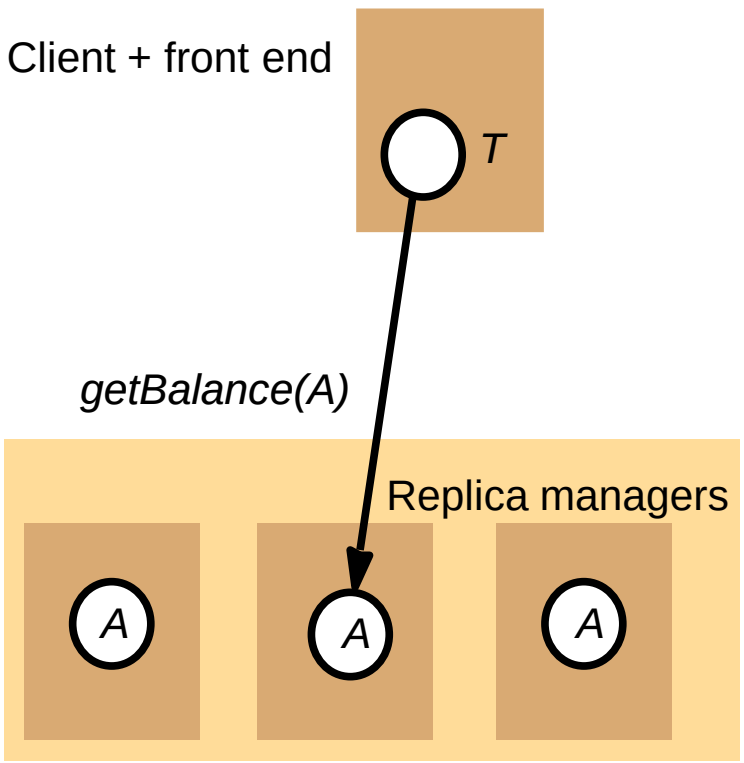


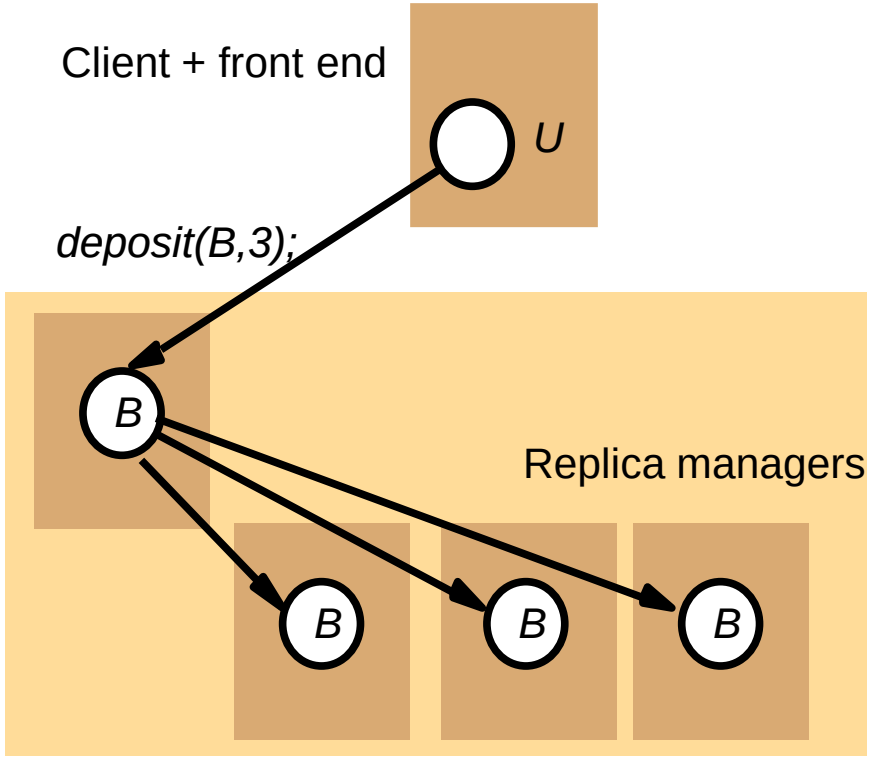
Committed and tentative updates in Bayou



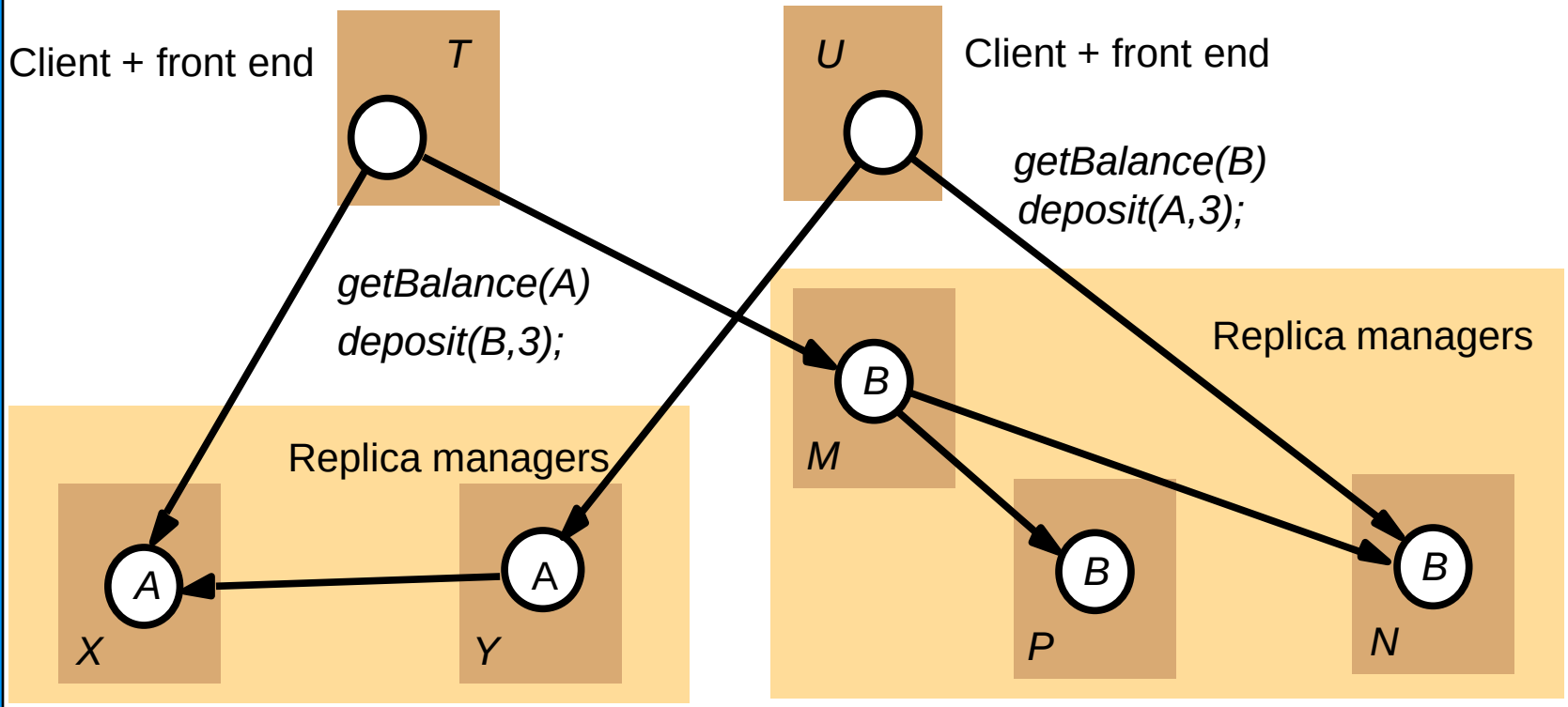
Tentative update t_i becomes the next committed update and is inserted after the last committed update c_N.

Transactions on replicated data

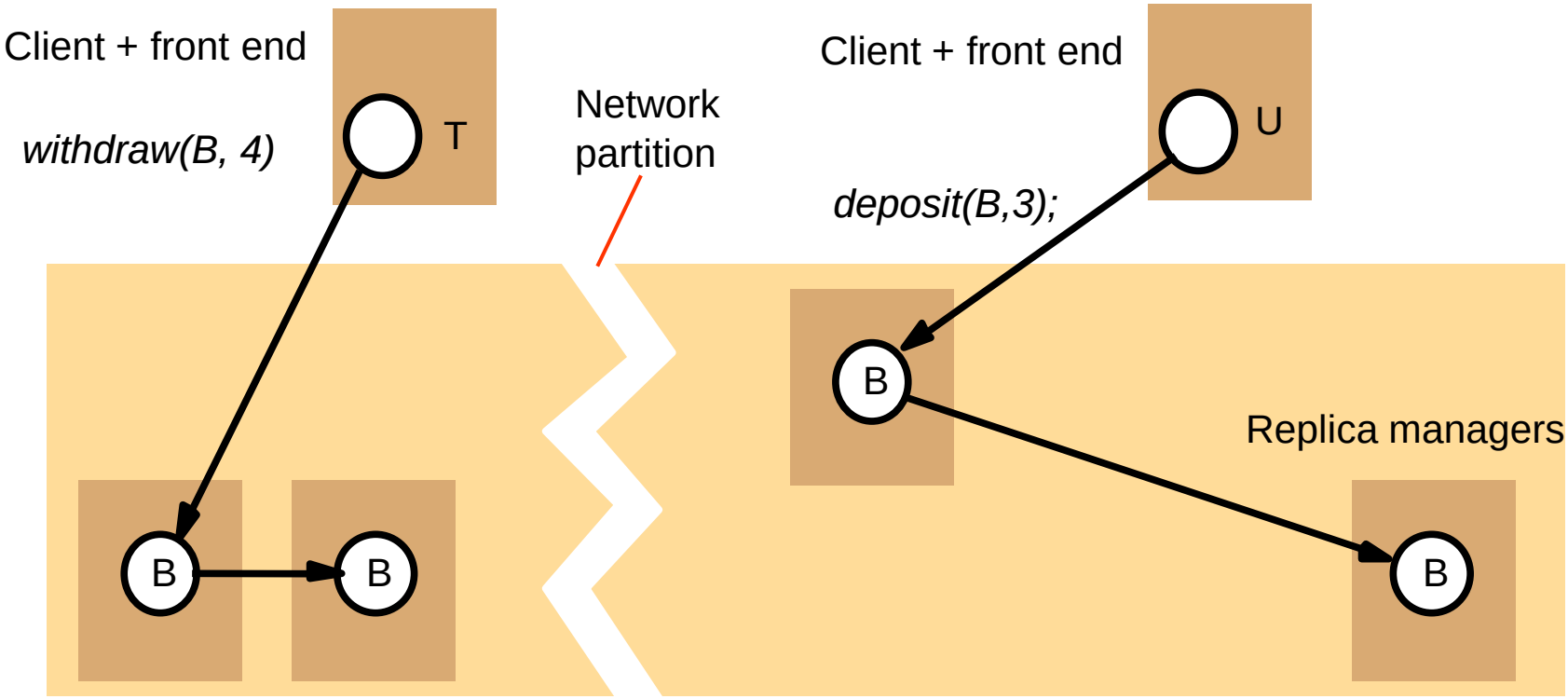




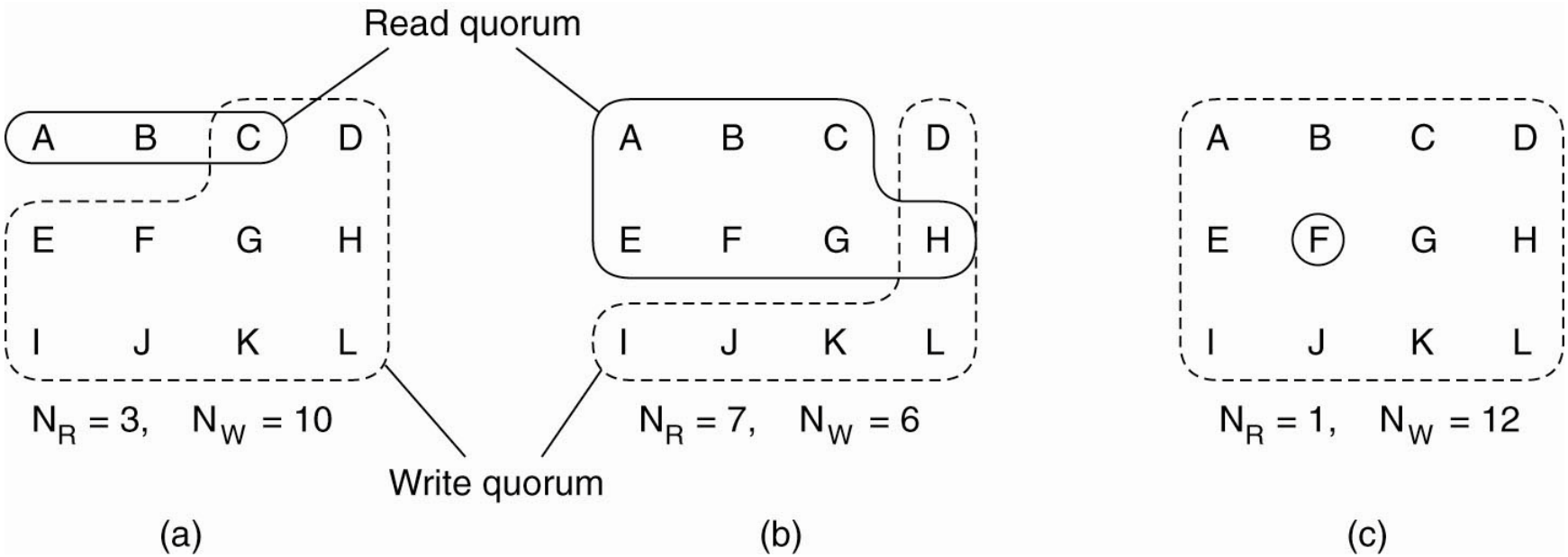
Available copies



Network partition



Quorum-Based Protocols



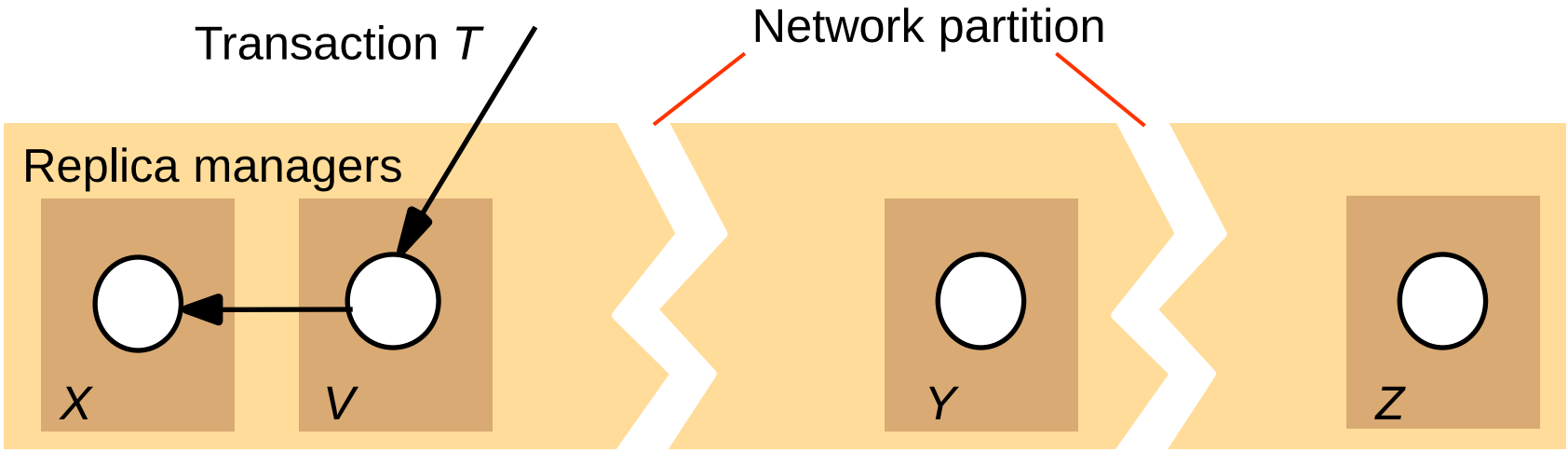
- Three examples of the voting algorithm. (a) A correct choice of read and write set. (b) A choice that may lead to write-write conflicts. (c) A correct choice, known as ROWA (read one, write all).

Gifford's quorum concensus examples

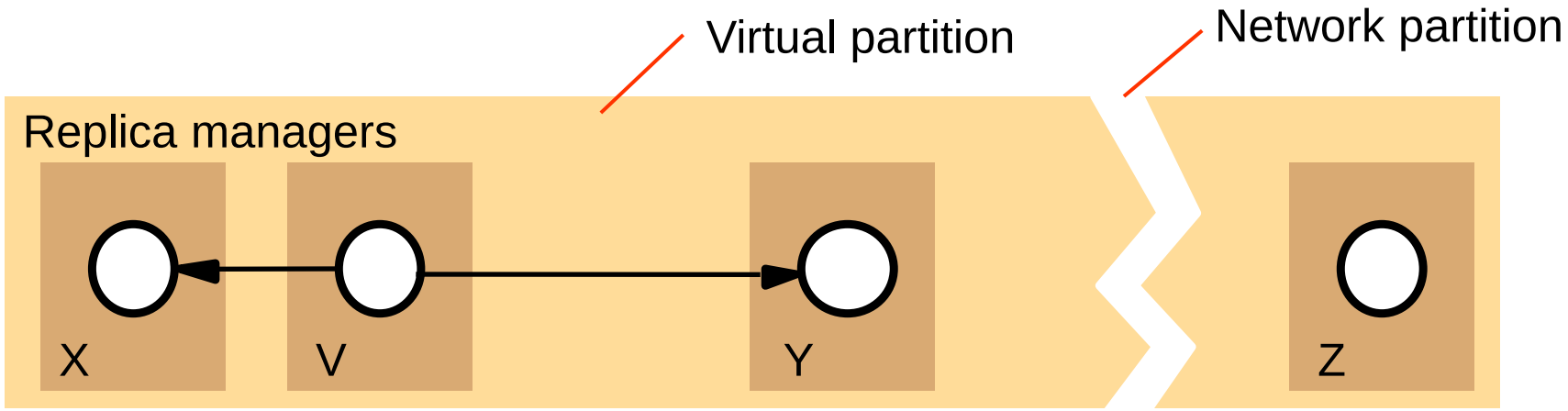
		Example 1	*Example 2*	*Example 3*
Latency (milliseconds)	Replica 1	75	75	75
	Replica 2	65	100	750
	Replica 3	65	750	750
Voting configuration	Replica 1	1	2	1
	Replica 2	0	1	1
	Replica 3	0	1	1
Quorum sizes	*R*	1	2	1
	W	1	3	3

Derived performance of file suite:				
Read	Latency	65	75	75
	Blocking probability	0.01	0.0002	0.000001
Write	Latency	75	100	750
	Blocking probability	0.01	0.0101	0.03

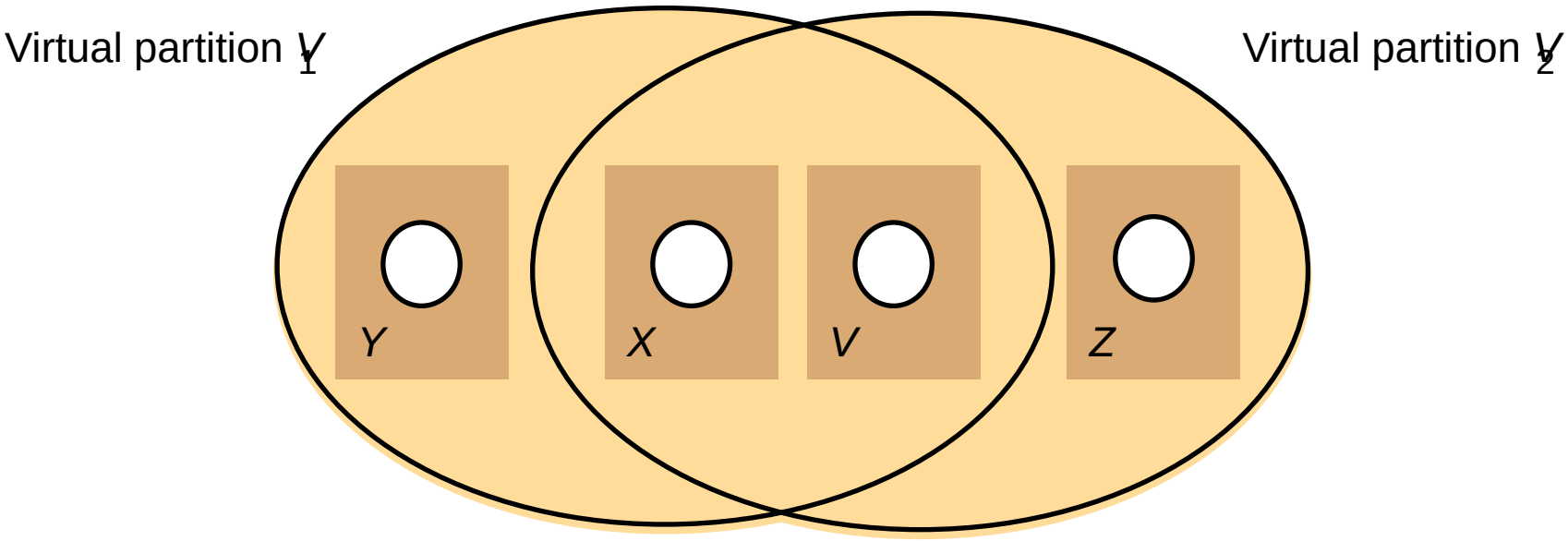
Two network partitions



Virtual partition



Two overlapping virtual partitions



Creating a virtual partition

Phase 1:

- The initiator sends a *Join* request to each potential member. The argument of *Join* is a proposed logical timestamp for the new virtual partition.
- When a replica manager receives a *Join* request, it compares the proposed logical timestamp with that of its current virtual partition.
  - If the proposed logical timestamp is greater it agrees to join and replies *Yes*;
  - If it is less, it refuses to join and replies *No*.

Phase 2:

- If the initiator has received sufficient *Yes* replies to have read and write quora, it may complete the creation of the new virtual partition by sending a *Confirmation* message to the sites that agreed to join. The creation timestamp and list of actual members are sent as arguments.
- Replica managers receiving the *Confirmation* message join the new virtual partition and record its creation timestamp and list of actual members.